Práctica 5 · Los filósofos van al teatro

Tiempo estimado: 2 h de laboratorio + unas 4 h de trabajo personal · Entrega voluntaria por Moodle hasta el domingo 18 de octubre a las 23:59 · Individual

Objetivo

Esta práctica junta todo lo que has hecho en el laboratorio hasta ahora en un solo proyecto: hilos que compiten por recursos (práctica 1), un servidor TCP (práctica 2) que dedica un hilo a cada cliente (práctica 3) y un protocolo binario con códigos de operación (práctica 4). Al terminarla sabrás coordinar hilos con cerrojos explícitos sin caer en un interbloqueo, limitar cuántos clientes atiende un servidor a la vez, proteger el estado que comparten varios hilos de servicio, y argumentar por escrito que tu programa no tiene condiciones de carrera ni interbloqueos. Ese argumento es parte de la práctica, igual que el código.

Material: proyecto Maven esqueleto `sd-prac5-teatro-esqueleto.zip` (Moodle) · IntelliJ IDEA (o VS Code / Eclipse) · JDK 17 o posterior · las transparencias «Hilos en Java» y «Sockets en Java» · el enunciado de la práctica 4.

La historia

Los cinco filósofos del problema de la cena se cansan de comer y en la sobremesa deciden ir al teatro con sus 95 colegas del congreso internacional de Filosofía Anaclética, que acaban de llegar a la ciudad. Se reparten los nombres de los colegas y deciden conectarse a la vez a la web del teatro para comprar entradas: 20 cada uno, para su grupo de colegas y para sí mismo.

El problema es que en la sala del congreso sólo hay tres ordenadores y tres teclados para los cinco. Un filósofo necesita un ordenador y un teclado, los dos a la vez, para comprar. Los filósofos son erráticos y cogen los dos en un orden cualquiera: unas veces primero un ordenador al azar y luego un teclado al azar, otras veces al revés. En cuanto tienen uno, intentan coger el otro.

El teatro Hiloserv Novengo tiene 50 butacas, numeradas del 0 al 49, y dos taquillas: sólo puede atender a dos clientes al mismo tiempo. Procesar cada venta le lleva un segundo. No puede vender más de 50 entradas ni puede vender la misma butaca dos veces.

Qué hay que construir

El cliente: los filósofos. Un programa con cinco hilos, uno por filósofo. Cada filósofo repite este ciclo hasta que tiene sus 20 entradas o el teatro le dice que se han agotado:

1. Coge un ordenador y un teclado. Si consigue el primero pero no el segundo, suelta el primero, espera un tiempo al azar y vuelve a empezar. Mientras espera no tiene nada en las manos.
2. Con los dos en su poder, se conecta al teatro. Si el teatro le dice que las taquillas están ocupadas, o no consigue conectar, suelta el ordenador y el teclado inmediatamente, espera un tiempo al azar y vuelve al paso 1. Si no consigue conectar cinco veces seguidas, se rinde y termina: el teatro no está en marcha o ha cerrado.
3. Si el teatro le atiende, compra entradas de una en una, hasta diez en esa visita. Después se despide, cierra la conexión y suelta el ordenador y el teclado. Para comprar la segunda tanda de diez tiene que volver a cogerlos.
4. Cuando el teatro le responde que las entradas se han agotado, o cuando ya tiene las 20, el filósofo dice quién es, cuántas entradas ha podido comprar y qué butacas son, y termina.

Cuando los cinco han terminado, el programa hace el recuento: cuántas entradas hay en total y si alguna butaca aparece dos veces.

El servidor: el teatro. Escucha en `localhost:9876`. Por cada cliente que admite crea un hilo «taquilla» que le atiende durante toda la conexión. Como hay dos taquillas, no puede haber más de dos clientes atendidos al mismo tiempo: al tercero que llegue le dice que las taquillas están ocupadas y cierra su conexión. Cada taquilla vende sobre el mismo patio de butacas: ninguna butaca se vende dos veces y nunca se venden más de 50. Cuando ha vendido todo, ya no queda nadie en las taquillas y lleva cinco segundos sin recibir a nadie, el teatro cierra y muestra el resumen de la venta.

El protocolo

Todo viaja con `DataInputStream` y `DataOutputStream`, como en la práctica 4: un código de operación (un `int`) y, detrás, la carga que indica la tabla. El teatro habla primero: nada más aceptar la conexión dice si atiende o no (así lo hacen también servidores clásicos como los de correo). El contrato completo está en la clase `Protocolo`, que comparten cliente y servidor.

Sentido	Código	Carga tras el código	Qué significa
servidor → cliente	`BIENVENIDO` (1)	`int` taquilla	le atiende la taquilla número n
servidor → cliente	`OCUPADO` (2)	(ninguna)	las dos taquillas están ocupadas; el servidor cierra a continuación
cliente → servidor	`COMPRAR` (10)	`int` filósofo	una entrada, por favor
servidor → cliente	`ENTRADA` (3)	`int` butaca	aquí tiene la entrada
servidor → cliente	`AGOTADAS` (4)	(ninguna)	no quedan entradas
cliente → servidor	`ADIOS` (11)	(ninguna)	el cliente cierra a continuación

Una visita normal es, por tanto: `BIENVENIDO 1`, y luego hasta diez veces `COMPRAR 3` seguido de `ENTRADA 17` (o de `AGOTADAS`), y por último `ADIOS`. La tabla es el protocolo: todos los valores viajan en ese orden, y tras cada mensaje, en los dos sentidos, se vacía el flujo con `flush()`.

El proyecto esqueleto

Abre el proyecto en IntelliJ IDEA (File → Open… sobre la carpeta que contiene `pom.xml`; es un proyecto Maven normal, así que VS Code o Eclipse sirven igual). El proyecto compila tal cual, pero no hace nada útil hasta que completes las seis tareas: hasta que las tareas 5 y 6 están hechas el cliente no termina nunca (páralo tú), y hasta que el teatro vende, tampoco cierra. Cada tarea es un comentario `TAREA N` en el código, con lo que hay que hacer paso a paso.

Fichero	Estado
`comun/Protocolo`	dado: los códigos, la carga de cada mensaje y las constantes del teatro; léelo primero
`comun/Registro`	dado: escribe una línea en la consola con la hora y quién habla
`servidor/Butacas`	**tarea 1**: `vender()`, que vende la primera butaca libre y es seguro frente a hilos
`servidor/Taquillas`	**tarea 2**: `tomar()` y `liberar()`, el control de cuántas taquillas están ocupadas
`servidor/Teatro`	el bucle de `accept()` y el cierre del teatro están escritos; **tarea 3**: admitir a un cliente o rechazarlo
`servidor/Taquilla`	la estructura del hilo, con sus `catch` y su `finally`, está escrita; **tarea 4**: la conversación con el cliente
`cliente/Recursos`	dado: los tres ordenadores y los tres teclados, cada uno un `ReentrantLock`
`cliente/Filosofo`	`run()` y `cogerDos()` dados; **tarea 5**: coger ordenador y teclado al azar; **tarea 6**: la visita al teatro
`cliente/Filosofos`	dado: arranca los cinco hilos, espera a que terminen y hace el recuento

Se ejecuta con dos procesos, como en las prácticas anteriores: primero `servidor.Teatro`, después `cliente.Filosofos` (una configuración de ejecución para cada uno). Con las ventas a un segundo, una ejecución completa dura alrededor de medio minuto; con el argumento de programa `--rapido`, el teatro vende sin esperar, lo que conviene para probar.

Cerrojos explícitos. Con `synchronized` un hilo que quiere un cerrojo ocupado espera hasta que se lo den, sin límite. Aquí eso no sirve: un filósofo con un ordenador en la mano no puede quedarse esperando un teclado para siempre mientras otro, con un teclado en la mano, espera un ordenador para siempre. Por eso los recursos son objetos `Lock` (`java.util.concurrent.locks.ReentrantLock`), que permiten intentar coger el cerrojo durante un tiempo limitado con `tryLock` y, si no se consigue, soltar lo que se tenía con `unlock`. El método `cogerDos`, que te damos hecho en `Filosofo`, hace eso mismo:

```
private boolean cogerDos(Lock primero, Lock segundo) throws InterruptedException {
    if (!primero.tryLock(ESPERA_CERROJO_MS, TimeUnit.MILLISECONDS)) {
        return false;
    }
    try {
        if (!segundo.tryLock(ESPERA_CERROJO_MS, TimeUnit.MILLISECONDS)) {
            primero.unlock();
            return false;
        }
    } catch (InterruptedException e) {
        primero.unlock();
        throw e;
    }
    return true;
}
```

Si devuelve `false`, el filósofo no tiene ningún cerrojo. Entonces espera un tiempo al azar (`esperaAlAzar()`, también dado) y vuelve a intentarlo. Ésa es la técnica de «soltar y esperar un tiempo aleatorio». Soltar a tiempo evita el interbloqueo cuando no se puede imponer un orden fijo para coger los recursos; esperar un tiempo distinto cada vez evita que dos filósofos repitan el mismo choque.

Comprobaciones y preguntas

1. Ejecuta el teatro y los filósofos varias veces. Al final, el recuento del cliente debe decir «50 entradas en total, 50 butacas distintas» y el resumen del teatro «50 butacas vendidas». Anota, en una de las ejecuciones, cuántas entradas ha conseguido cada filósofo y cuántas veces ha rechazado el teatro a alguien. Los repartos pueden ser muy desiguales: explica por qué un filósofo puede terminar con cero entradas sin que haya ningún error.
2. Tres filósofos consiguen a la vez ordenador y teclado y se conectan. El `accept()` del teatro devuelve un `Socket` para los tres: la conexión TCP del tercero se establece. ¿Por qué no puede el `ServerSocket` negarse a aceptarla? ¿En qué punto exacto del código, y con qué mensaje, se entera el tercer filósofo de que no le atienden? ¿Qué hace con el ordenador y el teclado en ese momento, y qué pasaría con los otros filósofos si se los quedara mientras espera?
3. Quita `synchronized` de `vender()` y añade dentro, entre comprobar que una butaca está libre y apuntarla como vendida, una llamada `pausa(5)` (el método `pausa` ya está en `Butacas`: es un paso lento en el peor sitio posible). Ejecuta el teatro con `--rapido` y los filósofos. Copia el recuento del cliente y la línea «cerrado: …» del teatro, y explica cómo puede ser que el teatro diga que ha vendido 50 butacas y los filósofos tengan más de 50 entradas. Vuelve a poner `synchronized` y quita la pausa. ¿Basta con que el recuento salga bien en diez ejecuciones para afirmar que no hay carrera? Razónalo con lo que viste en la práctica 4.
4. Argumenta que tu programa no puede quedarse en interbloqueo. Nombra qué condición necesaria para el interbloqueo deja de cumplirse gracias a `cogerDos`, y explica por qué. ¿Qué papel tiene la espera al azar? Describe una ejecución en la que, sin ella, dos filósofos pudieran repetir el mismo choque una y otra vez; a esa situación se la llama bloqueo activo (ingl. *livelock*).

5. El hilo de una taquilla llama a `setSoTimeout` sobre su socket, y su `finally` libera la taquilla pase lo que pase. Mata el proceso de los filósofos a mitad de una compra (el botón de parar de IntelliJ) y observa la consola del teatro. ¿Qué excepción salta en la taquilla, y en qué instrucción? ¿Qué habría pasado sin el `finally`? Ahora imagina un cliente que se conecta, recibe `BIENVENIDO` y no dice nada más: ¿cuánto tarda el teatro en recuperar esa taquilla, y por qué hace falta el plazo? Guarda esta respuesta: en el tema 2 viste que un plazo de espera es lo único que un proceso tiene para sospechar de otro.
6. Escribe, byte a byte, lo que viaja por el cable en una compra completa: la petición `COMPRAR` del filósofo 3 y la respuesta `ENTRADA` con la butaca 17. Para cada campo, su tipo Java, su valor, su tamaño en bytes y el orden en el cable, más el tamaño total de cada mensaje. Es el tipo de pregunta que el examen de prácticas hace sobre protocolos como éste.

Entrega

Entrega voluntaria, sin nota, hasta el domingo 18 de octubre a las 23:59. Si quieres que te la corrija, sube a la tarea de Moodle de la práctica 5 un zip del proyecto con tu código y un fichero `RESPUESTAS.md` (o PDF) con las respuestas a las preguntas 1 a 6. La solución se publica el lunes 19 de octubre. Esta práctica cierra el laboratorio 1: el examen de prácticas PR-EX-1, previsto para el martes 20 de octubre (la fecha se confirma en Moodle), examina Java, TCP y marshalling, que es todo lo que has usado aquí.

Nota de la edición 2026-27

Basado en un enunciado de examen de Programación de Servicios y Procesos, IES Clara del Rey (2021-22), del mismo autor. El enunciado original usaba un protocolo de líneas de texto; esta edición lo sustituye por el protocolo binario con códigos de operación de la práctica 4 y añade el esqueleto, el cierre ordenado del teatro y las preguntas. Los identificadores están en castellano, como en el resto de las prácticas de este curso.